Static Analysis Project

Detecting and Remediating Reflected XSS and SQL Injection
in AssignmentServlet.java using Semgrep

Course: ITIS 4221/5221

Tool: Semgrep (taint-mode custom rules)

Target: assignment/AssignmentServlet.java

1. Overview

This report documents a static analysis exercise on **AssignmentServlet.java**, an intentionally vulnerable Java servlet. Two custom Semgrep taint-mode rules were written to detect three vulnerabilities: two instances of reflected Cross-Site Scripting (XSS) and one instance of SQL Injection (SQLi). After confirming detection, each vulnerability was patched and the scan was re-run to confirm zero remaining findings.

Three endpoints were in scope:

• **/asmt/comment** — reflected XSS (comment author/text reflected into HTML)
• **/asmt/search** — reflected XSS (search query reflected into HTML via helper builders)
• **/asmt/userByEmail** — SQL injection (email parameter concatenated into a SQL query)

2. Vulnerable Code

2.1 Reflected XSS — /asmt/comment

The **submitComment** method reads the *author* and *text* request parameters and inserts them directly into the HTML response without encoding. The *heading* string concatenates the raw *author* value, and the raw *text* value is passed straight into **Views.thankYouSections()**, which flows into **Views.layout()** and is written to the response via **ok()** → **writeHtml()** → **out.println()**. Because neither value is HTML-escaped, an attacker-supplied payload such as *<script>alert(1)</script>* in either parameter would execute in the victim's browser.

```
// VULNERABLE: stores and reflects raw data back into HTML without encoding
// TODO (fix): encode author/text before building the thank-you sections
private void submitComment(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String author = param(req, "author");
  String text = param(req, "text");
  addComment(new Comment(author, text));
  String heading = "Thanks, " + author;
  String html = Views.layout("Submitted (VULN)", Views.thankYouSections(heading, text));
  ok(resp, html);
}
```

Figure 1 — Vulnerable submitComment() method

2.2 Reflected XSS — /asmt/search

The **search** method reads the *q* parameter and passes it, unescaped, into **Views.searchSections()** and **Views.renderItems()**, both of which embed the raw string into HTML (e.g. inside an <h1> heading and repeated list items). The result is written back via **ok()** with no sanitization, giving an attacker the same reflected-XSS primitive as above.

```java
// XSS VULNERABLE: reflects raw query param into HTML builders
// TODO (fix): ensure query is encoded before adding to sections/list items
private void search(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String q = param(req, "q");
  ok(resp, Views.layout("Search (VULN)", Views.searchSections(q, Views.renderItems(q))));
}
```

Figure 2 — Vulnerable search() method

2.3 SQL Injection — /asmt/userByEmail

The **userByEmail** method reads the *email* parameter and passes it to **SqlTemplates.lookupByEmail()**, which builds a SQL string via plain concatenation (*"email = '" + value + "'"*). The resulting string is executed directly with **Statement.executeQuery()**. Because the email value is never separated from the SQL syntax, an attacker can break out of the intended query — e.g. supplying *' OR '1'='1* — to alter the query's logic or exfiltrate data.

```java
// SQLi VULNERABLE: concatenates raw email into SQL and executes
// TODO (fix): use a PreparedStatement instead of building SQL with string concatenation
private void userByEmail(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String email = param(req, "email");
  String sql = SqlTemplates.lookupByEmail(email); // tainted concat via helper
  try {
    Connection conn = getConnection();
    Statement st = conn.createStatement();
    ResultSet rs = st.executeQuery(sql); // tainted sink
    ok(resp, Views.layout(
        "UserByEmail (VULN)",
        Views.noticeSections("Lookup", "Queried by email.")));
    rs.close();
    st.close();
    conn.close();
  } catch (SQLException e) {
    ok(resp, Views.layout(
        "UserByEmail (VULN)",
        Views.noticeSections("Lookup", "DB error (expected in assignment)")));
  }
}
```

Figure 3 — Vulnerable userByEmail() method

3. Semgrep Detection Rules

Two custom taint-mode rules were written in **assignment_rules_starter.yaml**: one for reflected XSS (**asmt-java-xss**) and one for SQL injection (**asmt-java-sqli**). Both rules use *request.getParameter(...)* / the project's *param(req, ...)* wrapper as taint sources. Because taint does not automatically cross helper-method boundaries in Semgrep's taint engine, **pattern-propagators** were added to explicitly track taint through the project's own helper methods (*Views.thankYouSections*, *Views.searchSections*, *Views.layout*, and *SqlTemplates.lookupByEmail*). The XSS rule treats *out.println*, *response.getWriter().println/print*, and the project's *ok()*, *bad()*, and *writeHtml()* wrappers as sinks, and recognizes *StringEscapeUtils.escapeHtml4* (and the project's local *escape()* wrapper) as sanitizers. The SQLi rule treats *Statement.executeQuery/executeUpdate/execute* and *Connection.prepareStatement* as sinks.

3.1 Scan on Vulnerable Code

Running **semgrep --dataflow-traces --config assignment_rules_starter.yaml assignment** against the original code produced **3 findings** (3 blocking): two *asmt-java-xss* findings (comment endpoint at line 90, search endpoint at line 227) and one *asmt-java-sqli* finding (userByEmail endpoint at line 238), each with a full dataflow trace from the tainted *param(req, ...)* source to the sink.

```
asaltiwari@MacBookAir project % semgrep --dataflow-traces --config assignment_rules_starter.yaml assignment

  Semgrep CLI

⁝ Loading rules...                                  Scanning 1 file (only git-tracked) with 2 Code rules:

  CODE RULES
  Scanning 1 file with 2 java rules.

  SUPPLY CHAIN RULES

  No rules to run.


  PROGRESS

  ──────────────────────────────────── 100% 0:00:00


  3 Code Findings

    assignment/AssignmentServlet.java
   ❯❯❱ asmt-java-xss
          ❮❮ Blocking ❯❯
          Reflected XSS: tainted user input reaches HTML response

           90┆ ok(resp, html);


          Taint comes from:

           85┆ String author = param(req, "author");


          Taint flows through these intermediate variables:

           85┆ String author = param(req, "author");

           88┆ String heading = "Thanks, " + author;

           89┆ String html = Views.layout("Submitted (VULN)",
               Views.thankYouSections(heading, text));


                This is how taint reaches the sink:

           90┆ ok(resp, html);


             ⁝┆----------------------------------------
          227┆ ok(resp, Views.layout("Search (VULN)",
               Views.searchSections(q, Views.renderItems(q))));


          Taint comes from:
```

Figure 4 — Semgrep run on vulnerable code (part 1)

```
          Taint comes from:

          226¦ String q = param(req, "q");


          Taint flows through these intermediate variables:

          226¦ String q = param(req, "q");


                This is how taint reaches the sink:

          227¦ ok(resp, Views.layout("Search (VULN)",
               Views.searchSections(q, Views.renderItems(q))));


            ⋮¦----------------------------------------

  ❯❯❯ asmt-java-sqli
        ❮❮ Blocking ❯❯
        SQL injection: tainted request parameter flows into SQL
        execution

        238¦ ResultSet rs = st.executeQuery(sql); // tainted
             sink


        Taint comes from:

        233¦ String email = param(req, "email");


        Taint flows through these intermediate variables:

        233¦ String email = param(req, "email");

        234¦ String sql = SqlTemplates.lookupByEmail(email);
             // tainted concat via helper


              This is how taint reaches the sink:

        238¦ ResultSet rs = st.executeQuery(sql); // tainted
             sink



┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
 • Findings: 3 (3 blocking)
 • Rules run: 2
 • Targets scanned: 1
 • Parsed lines: ~100.0%
 • No ignore information available
Ran 2 rules on 1 file: 3 findings.
asaltiwari@MacBookAir project %
```

Figure 5 — Semgrep run on vulnerable code (part 2, continued)

4. Remediation

4.1 Fix — /asmt/comment

Both *author* and *text* are now passed through the project's **escape()** helper (which wraps *StringEscapeUtils.escapeHtml4*) before being placed into the response markup. This HTML-encodes special characters (e.g. <, >, &) so any injected markup is rendered as inert text rather than executed as HTML/JavaScript.

```
private void submitComment(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String author = param(req, "author");
  String text = param(req, "text");
  addComment(new Comment(author, text));
  String heading = "Thanks, " + escape(author);
  String html = Views.layout("Submitted (FIXED)", Views.thankYouSections(heading, escape(text)));
  ok(resp, html);
}
```

Figure 6 — Patched submitComment() method

4.2 Fix — /asmt/search

The query parameter is escaped once into *safeQ* immediately after being read, and the escaped value is reused for both the heading and the rendered result items, ensuring no raw user input reaches the HTML output.

```
private void search(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String q = param(req, "q");
  String safeQ = escape(q);
  ok(resp, Views.layout("Search (FIXED)", Views.searchSections(safeQ, Views.renderItems(safeQ))));
}
```

Figure 7 — Patched search() method

4.3 Fix — /asmt/userByEmail

The vulnerable string-concatenation helper (*SqlTemplates.lookupByEmail*) was replaced with a parameterized **PreparedStatement**. The SQL text now uses a literal *?* placeholder (*"SELECT id, email FROM users WHERE email = ?"*), and the user-supplied email is bound separately via *ps.setString(1, email)*. This guarantees the database driver treats the email strictly as data, never as part of the SQL syntax, eliminating the injection vector regardless of what characters the value contains.

```java
private void userByEmail(HttpServletRequest req, HttpServletResponse resp) throws IOException {
  String email = param(req, "email");
  String sql = "SELECT id, email FROM users WHERE email = ?";
  try {
    Connection conn = getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, email);
    ResultSet rs = ps.executeQuery();
    ok(resp, Views.layout(
        "UserByEmail (FIXED)",
        Views.noticeSections("Lookup", "Queried by email.")));
    rs.close();
    ps.close();
    conn.close();
  } catch (SQLException e) {
    ok(resp, Views.layout(
        "UserByEmail (FIXED)",
        Views.noticeSections("Lookup", "DB error (expected in assignment)")));
  }
}
```

Figure 8 — Patched userByEmail() method

5. Verification — Rescan After Fix

The same command was re-run against the patched file: **semgrep --dataflow-traces --config assignment_rules_starter.yaml assignment**. The scan reported **0 findings (0 blocking)** across both rules, confirming that the sanitizer (*escape()*) now sits between every taint source and the HTML sinks for the XSS rule, and that the SQLi rule no longer detects any tainted data reaching a SQL execution sink, since the query is fully parameterized.

```
asaltiwari@MacBookAir project % semgrep --dataflow-traces --config assignment_rules_starter.yaml assignment

┌─────────────┐
│ Semgrep CLI │
└─────────────┘

Scanning 1 file (only git-tracked) with 2 Code rules:

  CODE RULES
  Scanning 1 file with 2 java rules.

  SUPPLY CHAIN RULES

  No rules to run.


  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00


┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 2
 • Targets scanned: 1
 • Parsed lines: ~100.0%
 • No ignore information available
Ran 2 rules on 1 file: 0 findings.

✨ If Semgrep missed a finding, please send us feedback to let us know!
   See https://semgrep.dev/docs/reporting-false-negatives/
asaltiwari@MacBookAir project %
```

Figure 9 — Semgrep run on patched code — 0 findings